



**HELLEN NUNES DOS SANTOS SOUZA**

Ciência de Dados para Negócios — FATEC Votorantim

Gestão Financeira, Operações & Análise de Dados |

CPA ANBIMA

hellen.souza5@aluno.cps.sp.gov.br | (15) 99698-3992

Sorocaba – SP     

# Relatório Técnico: Análise de Risco de Crédito e Inadimplência

**Projeto:** Credit Risk & Default Analytics

**Repositório:** [Analise-de-Risco-de-Credito](#)

**Linguagem:** Python 3 | **Técnica:** Aprendizado de Máquina Supervisionado (Random Forest)

---

## 1. Contexto e Problema de Negócio

No mercado financeiro e bancário, antecipar o default (inadimplência) é indispensável para controlar o risco da carteira, calibrar limites de crédito e reduzir a Provisão para Créditos de Liquidação Duvidosa (PDD).

Este projeto cria uma esteira preditiva completa: simulação de dados transacionais, definição de regras de negócio, separação analítica, treinamento de modelo e extração dos fatores de risco que mais influenciam a tomada de decisão.

---

## 2. Explicação Passo a Passo (Linha por Linha)

### Bloco 1: Importação de Bibliotecas

Python

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score
```

- `import pandas as pd`: Fornece a estrutura de DataFrame para organizar, limpar e manipular tabelas de dados financeiros.

- `import numpy as np`: Biblioteca de computação matemática utilizada para geração de números aleatórios e operações vetorizadas.
- `from sklearn.model_selection import train_test_split`: Função do Scikit-Learn que divide o conjunto de dados em amostras separadas de treino e teste.
- `from sklearn.ensemble import RandomForestClassifier`: Algoritmo baseado em ensemble de árvores de decisão que votam em conjunto para prever se o cliente se tornará inadimplente.
- `from sklearn.metrics import classification_report, roc_auc_score`: Ferramentas estatísticas para mensurar precisão, revocação e a separabilidade geral do modelo (curva ROC).

---

## Bloco 2: Simulação da Carteira de Clientes

Python

```
np.random.seed(42)
n_records = 1500
```

- `np.random.seed(42)`: Fixa a semente pseudoaleatória, garantindo que os dados gerados e os resultados sejam idênticos em qualquer máquina.
- `n_records = 1500`: Estabelece o volume de contratos simulados na amostra.

Python

```
data = pd.DataFrame({
    'idade': np.random.randint(21, 65, n_records),
    'renda_mensal': np.random.normal(5500, 2000, n_records).round(2),
    'score_credito': np.random.randint(300, 850, n_records),
    'taxa_utilizacao_limite': np.random.uniform(0.1, 1.0, n_records).round(2),
    'dias_atraso_historico': np.random.poisson(2, n_records),
})
```

- `data = pd.DataFrame({...})`: Constrói a tabela tabular contendo as variáveis de crédito.
- 
- `'idade': np.random.randint(21, 65, n_records)`: Simula idades inteiras entre 21 e 64 anos (população economicamente ativa).
-

- `'renda_mensal': np.random.normal(5500, 2000, n_records).round(2)`: Cria uma curva normal de renda com média de R\$ 5.500,00 e desvio-padrão de R\$ 2.000,00.
- 
- `'score_credito': np.random.randint(300, 850, n_records)`: Simula pontuações padronizadas de birôs de crédito (como Serasa/Boa Vista).
- 
- `'taxa_utilizacao_limite': np.random.uniform(0.1, 1.0, n_records).round(2)`: Representa a porcentagem do limite concedido que está comprometida (de 10% a 100%).
- 
- `'dias_atraso_historico': np.random.poisson(2, n_records)`: Modela eventos raros via distribuição de Poisson com média de 2 atrasos pontuais.
- 

---

### Bloco 3: Regra de Negócio Sintética (Target de Risco)

Python

```
prob_default = (
    (data['score_credito'] < 500) * 0.4 +
    (data['taxa_utilizacao_limite'] > 0.8) * 0.3 +
    (data['dias_atraso_historico'] > 3) * 0.3
)
data['inadimplente'] = (np.random.rand(n_records) < prob_default).astype(int)
```

- `(data['score_credito'] < 500) * 0.4`: Adiciona 40% de probabilidade de default para clientes com score baixo.
- 
- `(data['taxa_utilizacao_limite'] > 0.8) * 0.3`: Adiciona 30% de risco se o cliente utiliza mais de 80% do seu limite.
- 
- `(data['dias_atraso_historico'] > 3) * 0.3`: Adiciona 30% de risco para clientes com mais de 3 dias recorrentes de atraso.
- 
- `(np.random.rand(n_records) < prob_default).astype(int)`: Gera a coluna-alvo binária: 0 para cliente adimplente e 1 para inadimplente.
- 

---

### Bloco 4: Divisão dos Dados em Treino e Teste

Python

```
X = data.drop(columns=['inadimplente'])
y = data['inadimplente']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

- `X = data.drop(...)`: Isola as características explicativas do tomador.
- 
- `y = data['inadimplente']`: Seleciona a variável que o algoritmo precisa aprender a prever.
- 
- `test_size=0.25`: Reserva 25% dos dados (375 clientes) exclusivamente para teste, mantendo 75% (1.125 clientes) para treino.
- 
- `stratify=y`: Garante a mesma proporção de inadimplentes em ambos os conjuntos, evitando desequilíbrios.
- 

---

## Bloco 5: Treinamento e Inferência do Modelo

Python

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

- `RandomForestClassifier(n_estimators=100, ...)`: Configura uma floresta de 100 árvores de decisão.
- `model.fit(X_train, y_train)`: Executa o treinamento e aprende os padrões que levam ao default.
- `y_pred = model.predict(X_test)`: Gera o rótulo final estimado (0 ou 1).
- `y_proba = model.predict_proba(X_test)[:, 1]`: Extrai a probabilidade contínua (ex.: 0.82 = 82% de risco de atraso), necessária para calcular a ROC-AUC e calibrar réguas de cobrança.

---

## 3. Avaliação de Desempenho e Resultados Obtidos

Resultados consolidados obtidos na base de teste:

| Métrica                           | Classe 0<br>(Adimplente) | Classe 1<br>(Inadimplente) | Geral /<br>Ponderado |
|-----------------------------------|--------------------------|----------------------------|----------------------|
| <b>Precisão<br/>(Precision)</b>   | 83%                      | 54%                        | 76%                  |
| <b>Sensibilidade<br/>(Recall)</b> | 88%                      | 43%                        | 77%                  |
| <b>F1-Score</b>                   | 0.85                     | 0.48                       | 0.76                 |
| <b>Acurácia Geral</b>             | —                        | —                          | <b>77%</b>           |
| <b>ROC-AUC Score</b>              | —                        | —                          | <b>0.8537</b>        |

- **Interpretação da ROC-AUC (0.8537):** O valor superior a 0.85 demonstra excelente capacidade do modelo de distinguir quem paga em dia de quem entra em atraso.

---

#### 4. Ranking de Variáveis de Risco (Feature Importance)

Peso de cada indicador determinado pelo algoritmo:

- **Score de Crédito (36,04%):** É o preditor de maior relevância; resume o comportamento histórico de quitação de dívidas.
- **Taxa de Utilização de Limite (21,61%):** Mede o estresse financeiro imediato; tomadores que usam o limite no teto apresentam risco iminente.
- **Renda Mensal (16,84%):** Capacidade financeira de absorver oscilações de despesas.
- **Idade (14,30%):** Estabilidade profissional e maturidade de consumo.

- **Dias de Atraso Histórico (11,22%):** Histórico interno de pontualidade.

---

## 5. Aplicações Práticas de Negócio

- **Ajuste de Cut-off (Ponto de Corte):** Em cenários de crise, a instituição financeira pode calibrar o ponto de corte para priorizar o *Recall* de inadimplentes, rejeitando operações de maior volatilidade mesmo que reduza ligeiramente a concessão total.
- 
- **Ações Preventivas de Cobrança:** Clientes com utilização do limite acima de 80% e score mediano podem entrar automaticamente em canais de renegociação prévia antes do vencimento da fatura.